

Metodología de Desarrollo de Proyecto IT: API RESTful

IDEACIÓN Y ESPECIFICACIÓN

Escribir un documento que analice el proyecto, identificando:

Ideación y requerimientos del negocio:

- El problema que resuelve la Web API y por qué sería valioso resolverlo
- Realizar un análisis de mercado: ¿Quiénes serían los usuarios concretos? ¿Es un mercado global o local? ¿Qué competidores hay? ¿Es rentable económicamente?

Análisis de requerimientos del usuario:

- Describir casos de uso principales, e historias de usuario importantes. Definir requerimientos funcionales (qué hace).
- Definir requerimientos no funcionales (escalabilidad, seguridad).

DISEÑO

MVP:

- Diseño de un MVP (Minimum Viable Product): Identificar las funciones básicas e imprescindibles que debe tener la API.
- UI: Dibujar una interfaz de usuario simple

Modelado:

- Hacer UML conceptual del proyecto con los componentes de software y su interacción (cliente, web api, base de datos), y UML de clases del proyecto, tratando de aplicar los patrones de diseño que vimos (u otros, si encuentran)

Crear un drive del proyecto, compartirlo con los docentes (sblas@imal.unl.edu.ar, rmatuk@dc.uba.ar) y subir la documentación.

IMPLEMENTACION

Prototipado:

- Escribir algún esqueleto del código (o sea, todo vacío, pero que tenga los nombres de las clases, y funciones stub)
- Escribir algunos tests para el esqueleto anterior (TDD)

Crear un repositorio en GitHub, compartirlo con los docentes, y subir el código.

1) Ideación y requerimientos del negocio

Esta es una aplicación Web/API que permite a los usuarios construir combos de comida personalizables (base + ingredientes + postre + bebida), recibir sugerencias de locales donde comprar esos ítems según gustos, disponibilidad y precio, y finalizar una orden o reservar para pickup/delivery. Aporta personalización, simplicidad al pedir y descubrimiento de comercios afines.

1.1 Problema que resuelve y valor

Problemas actuales

- Menús rígidos: muchas cadenas ofrecen combos fijos que no cubren preferencias o gustos específicos.
- Dificultad para encontrar locales que ofrezcan exactamente los ingredientes o alternativas deseadas.
- Usuarios que desean tener control al realizar los pedidos (tamaños, sustituciones, adiciones) sin procesos complejos.

Valor de resolverlo

- Mayor satisfacción del cliente al obtener exactamente lo que quiere.
- Aumento de ventas para locales por ventas incrementales por upsell (producto superior o más completo que el original) de ingredientes.
- Diferenciación para marketplaces/locales que integren la API (personalización + recomendaciones).
- Reducción del abandono de carrito al ofrecer alternativas y combinaciones claras.
- Datos de preferencias que permiten ofertas personalizadas y optimización de inventarios.
- Alternativa al método de pedido propuesto por otras marcas como PedidosYa y Rappi.

1.2 Análisis de mercado

Usuarios concretos

- Consumidores finales que buscan personalizar pedidos (jóvenes, profesionales, familias, personas con restricciones alimentarias o gustos específicos).
- Restaurantes y dark kitchens (establecimientos que operan online con envíos a domicilios sin espacios físicos de atención) interesados en ofrecer combos dinámicos.
- Cadenas de retail y supermercados interesados en kits combinables.

Cobertura geográfica

- Modelo inicialmente local o regional (integración con comercios locales para disponibilidad/stock)

Competidores / alternativas

- Apps de delivery / marketplaces (Glovo, Rappi, Uber Eats, PedidosYa) ofrecen menús, pero suelen tener combos fijos y opciones limitadas de personalización por local.
- Sistemas configuradores de menú internos de restaurantes.
- Algunos sitios web de “build your own” (pizzerías, ensaladas) con experiencias cerradas.

Rentabilidad

- Modelos de ingresos posibles: comisiones por venta/orden, suscripción Premium para locales, fees por integración/API, publicidad/promociones dirigidas, datos agregados/anónimos para inteligencia de mercado.
- Es rentable si se alcanza masa crítica de locales y usuarios (efecto de red: más locales → mejor oferta → más usuarios → más conversiones).

2) Análisis de requerimientos del usuario

Suposiciones: Primera versión (MVP) cubrirá creación de combos, persistencia de combos, búsqueda de locales compatibles, visualización de precios y checkout básico (integración con local o redirección).

2.1 Casos de uso principales

1. **Crear combo personalizado**
 - Usuario selecciona base (ej. Sandwich, pizza, ensalada), añade/saca ingredientes, selecciona acompañamiento, postre y bebida, ajusta tamaños y cantidades.
2. **Guardar combo favorito**
 - Guardar combinación con nombre y preferencias.
3. **Recibir sugerencias de locales**
 - Dada la composición, el sistema sugiere locales que pueden preparar ese combo o que ofrecen los ítems/alternativas.
4. **Comparar precio y tiempo**
 - Ver precios por local y tiempos estimados (preparación/entrega).
5. **Checkout / Reserva**
 - Finalizar compra vía integración con local o generar orden para marketplace.

6. Filtrado por restricciones

- Buscar locales que cumplan con las restricciones dadas. Por ej: alergias (sin gluten), dietas (vegano), o preferencias (bajo sodio).

7. Recomendaciones personalizadas

- Motor sugiere ingredientes o combos basados en historial y calificaciones.

8. Administración del local

- Local gestiona catálogo de bases/ingredientes, precios, reglas de combinación, disponibilidad e inventario.

9. Analítica

- Local/administrador ve ventas, combos populares y datos agregados.

2.2 Historias de usuario (ejemplos)

- **HU1 (Consumidor):** "Como usuario, quiero combinar una base + ingredientes + bebida y ver qué locales pueden prepararlo para pedirlo ahora."
- **HU2 (Local):** "Como dueño de un restaurante, quiero definir reglas (ej. no combinar ingrediente X con Y) y precios por adición para que los pedidos sean válidos."
- **HU3 (App):** "Como integrador, quiero que consuman la Web API para mostrar combos y permitir checkout desde mi app."
- **HU4 (Consumidor):** "Quiero elegir comidas de bajo sodio"

2.3 Requerimientos funcionales (RF)

RF-1: Crear/editar/eliminar combo (base + ingredientes + postre + bebida).

RF-2: Validación de combos según reglas del local (ej. máximo 5 ingredientes, incompatibilidades).

RF-3: Búsqueda de locales compatibles por ubicación, tiempo, precio, y restricciones dietéticas.

RF-4: Mostrar desglose de precio (base + cada ingrediente + descuentos de combo).

RF-5: Guardar combos favoritos por usuario.

RF-6: Recomendar ingredientes y locales basados en historial y perfil.

RF-7: Checkout: crear orden y enviarla al local o marketplace

RF-8: soporte para pago externo (redirección) y para pagos in-app (si se implementa).

RF-9: API pública/privada para que terceros integren (endpoints: /combos, /locales, /ordenes, /usuarios, /inventario).

RF-10: Panel admin para locales — gestionar catálogo, reglas.

RF-11: Registro y login de usuarios (email, social login opcional).

RF-12: Ratings y feedback por combo/local.

2.4 Requerimientos no funcionales (RNF)

Escalabilidad

- **RNF-1:** Arquitectura desacoplada (microservicios o servicios modulares) que permita escalar componentes: motor de recomendaciones, catálogo/inventario, búsqueda/geolocalización.
- **RNF-2:** Diseño horizontalmente escalable; base de datos con particionado y/o sharding para catálogos y tráfico alto.
- **RNF-3:** Cache distribuida para resultados de búsqueda y combos frecuentes (Redis).
- **RNF-4:** Soporte de múltiples monedas e impuestos por región (para expansión internacional).
- **RNF-5:** Código modular, API versioning, documentación (OpenAPI/Swagger), y CI/CD con tests automatizados.

Seguridad

- **RNF-6:** Autenticación basada en OAuth2 / JWT; roles (usuario, local-admin, integrador, super-admin).
- **RNF-7:** Cumplimiento de privacidad para datos personales; políticas de retención y borrado.
- **RNF-8:** Datos personales minimizados; opciones para anonimizar datos de consumo antes de análisis.
- **RNF-9:** Backups regulares y pruebas de restore.

Accesibilidad

- **RNF-10:** UI accesible.

3) Diseño MVP (para la Web API y flujo)

3.1 Componentes principales

- **API Gateway / REST** — expone endpoints.
- **Servicio de Combos** — lógica para construir/validar combos, precios y reglas.

- **Catálogo / Inventario** — items, precios, disponibilidad por local.
- **Servicio de Locales** — perfil, ubicación, horarios, capacidades (qué puede preparar).
- **Motor de Recomendaciones** — sugerencias basadas en perfil/historial/compatibilidad.
- **Servicio de Búsqueda/Geolocalización** — búsqueda por proximidad y filtros.
- **Orquestador de Ordenes** — crea y envía órdenes a locales o a integradores de delivery.
- **Autenticación/Autorización** — OAuth2/JWT.
- **Data Warehouse / Analítica** — métricas agregadas.
- **Panel admin / Dashboard** — para locales y administración.

3.2 Prioridad y alcance para MVP

MVP (release 1) — foco en valor mínimo viable

1. Construcción de combo (interfaz simple + validación básica).
2. Catálogo por local (manualmente ingresado o CSV upload).
3. Búsqueda de locales por proximidad y disponibilidad básica.
4. Cálculo de precio y desglose.
5. Guardado de combos favoritos y perfil de usuario.
6. Endpoints públicos básicos y documentación (OpenAPI).
7. Panel simple para locales (CRUD de catálogo).

3.3 Diseño del MVP (Minimum Viable Product)

El **MVP** busca ofrecer las funciones **básicas y necesarias** para validar la idea y el flujo principal de la aplicación, sin desarrollar todas las funcionalidades completas del producto final.

Objetivo del MVP:

Validar que los usuarios disfruten crear combos personalizados y que los locales puedan ofrecerlos fácilmente.

Funciones básicas de la Web API

Módulo	Funcionalidad MVP	Descripción
Usuarios	Registro y autenticación básica	Permitir crear cuenta e iniciar sesión (JWT / token simple).
Catálogo de locales	CRUD de locales y sus productos	Registrar locales, sus bases, ingredientes y precios.
Construcción de combo	Crear y guardar combos personalizados	Permitir al usuario elegir base, ingredientes, bebida y acompañamiento.
Recomendaciones simples	Filtrado por cercanía o coincidencia de ingredientes	Sugerir locales que ofrezcan esos ítems o similares.
Checkout simulado	Crear orden (sin pago real)	Generar un resumen de pedido con local seleccionado.
Persistencia	Guardar datos en base relacional	Usuarios, locales, productos, combos y órdenes.
Seguridad básica	Autenticación + validación de permisos	Separar roles (usuario y local).

3.4. UI — Interfaz de usuario simple

Aquí se propone un **boceto textual (wireframe)** de la interfaz web principal.

Pantalla principal

```
+-----+
| LOGO ComboApp                                [Iniciar Sesión] [Registro] |
+-----+
| ¡Creá tu combo ideal!                         |
| [Agregar Comida +]                           |
| [Agregar Bebida +]                           |
| [Agregar Postre +]                           |
+-----+
| [Ver sugerencias de locales Cercanos][Guardar Combo Favorito] |
+-----+
```

Pantalla de usuario

```
+-----+
| Bienvenido!                                     |
+-----+
| Tus combo elegido:                             |
| - (Pan + lechuga + queso + bacon) + agua + cola |
+-----+
| Sugerencias de locales:                       |
| Local "BurgerMix"    - 2 km - $1650 - [Ver Detalles] |
| Local "BurgerBowl"   - 3 km - $1350 - [Ver Detalles] |
+-----+
| [Volver página anterior]                       |
+-----+
```

Pantalla “Detalles” Local seleccionado

```
+-----+
| Local: BurgerMix                               |
+-----+
| Productos disponibles:                         |
| Base: Pan brioche ($600)                       |
| Ingredientes: Lechuga ($100), Queso ($200), Bacon ($250) |
| Bebidas: Cola ($300), Agua ($200)              |
+-----+
| [Agregar al combo] [Volver página anterior] [Enviar pedido] |
+-----+
```

3.5 Modelado UML

A continuación, los diagramas conceptuales y de clases más relevantes para el MVP.

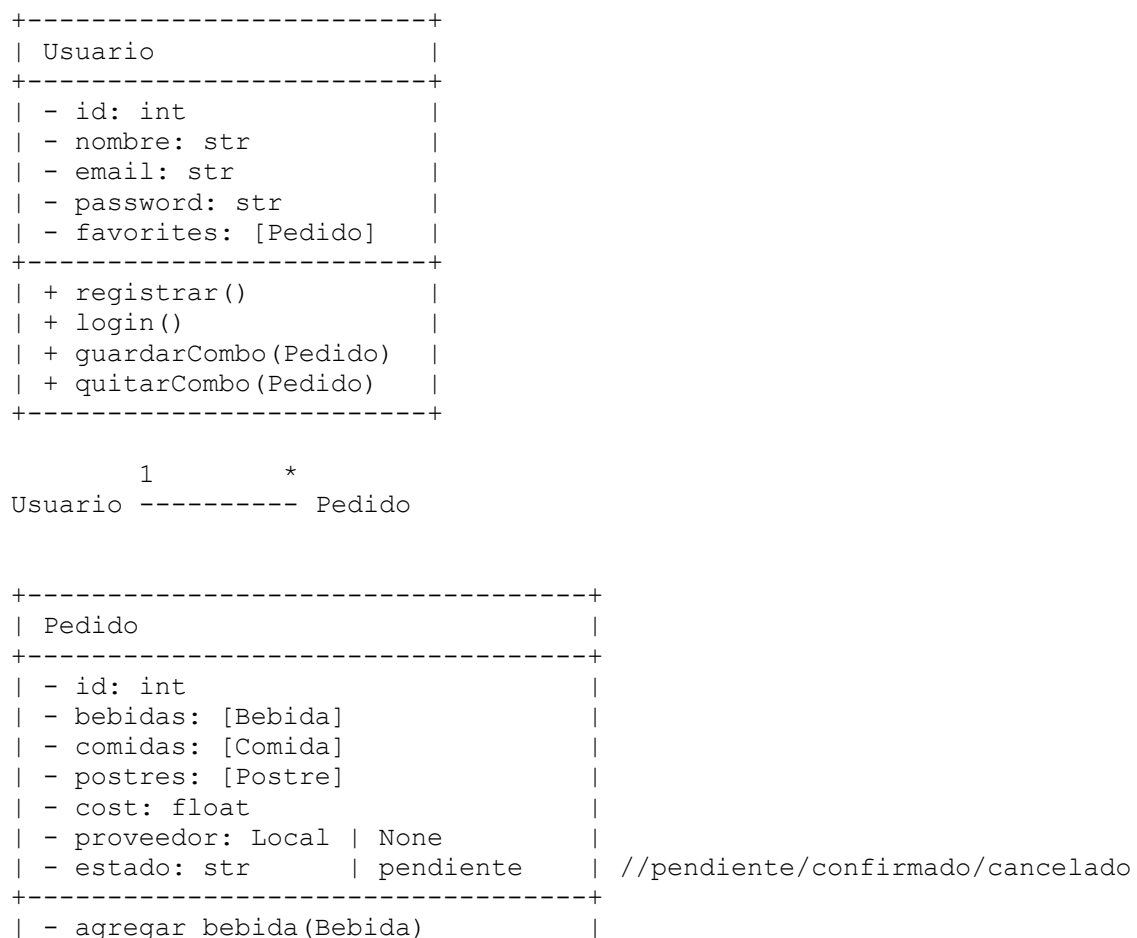
UML conceptual (arquitectura general)



Flujo principal:

1. El usuario crea un combo → el cliente envía datos al endpoint `/api/combos`.
2. La API valida y guarda el combo → consulta los locales compatibles en la BD.
3. Devuelve la lista de locales recomendados.
4. El cliente muestra las opciones y permite finalizar la orden

3.6. UML de clases del proyecto



```

| - agregar_comida(Comida)      |
| - agregar_postre(Postre)     |
| - mod_bebida(str)             |
| - mod_comida(str)             |
| - mod_postre(str)            |
| - eliminar_bebida(str)        |
| - eliminar_comida(str)        |
| - eliminar_postre(str)        |
| - calcularTotal()             |
| - setLocal(Local)             |
| - setEstado(pen./con./can.)   |
+-----+

```

```

      1      *
Pedido ----- Bebida
      1      *
Pedido ----- Comida
      1      *
Pedido ----- Postre

```

```

+-----+
| Comida |
+-----+
| - name: str |
| - cost: float |
+-----+

```

```

+-----+
| Bebida |
+-----+
| - name: str |
| - cost: float |
+-----+

```

```

+-----+
| Postre |
+-----+
| - name: str |
| - cost: float |
+-----+

```

```

+-----+
| Burger(Comida) |
+-----+
| - name: str |
| - cost: float |
| - amount: int |
| - pattyLevels: int |
| - bunType: str |
| - pattySize: str |
| - pattyStyle: str |
+-----+

```

```

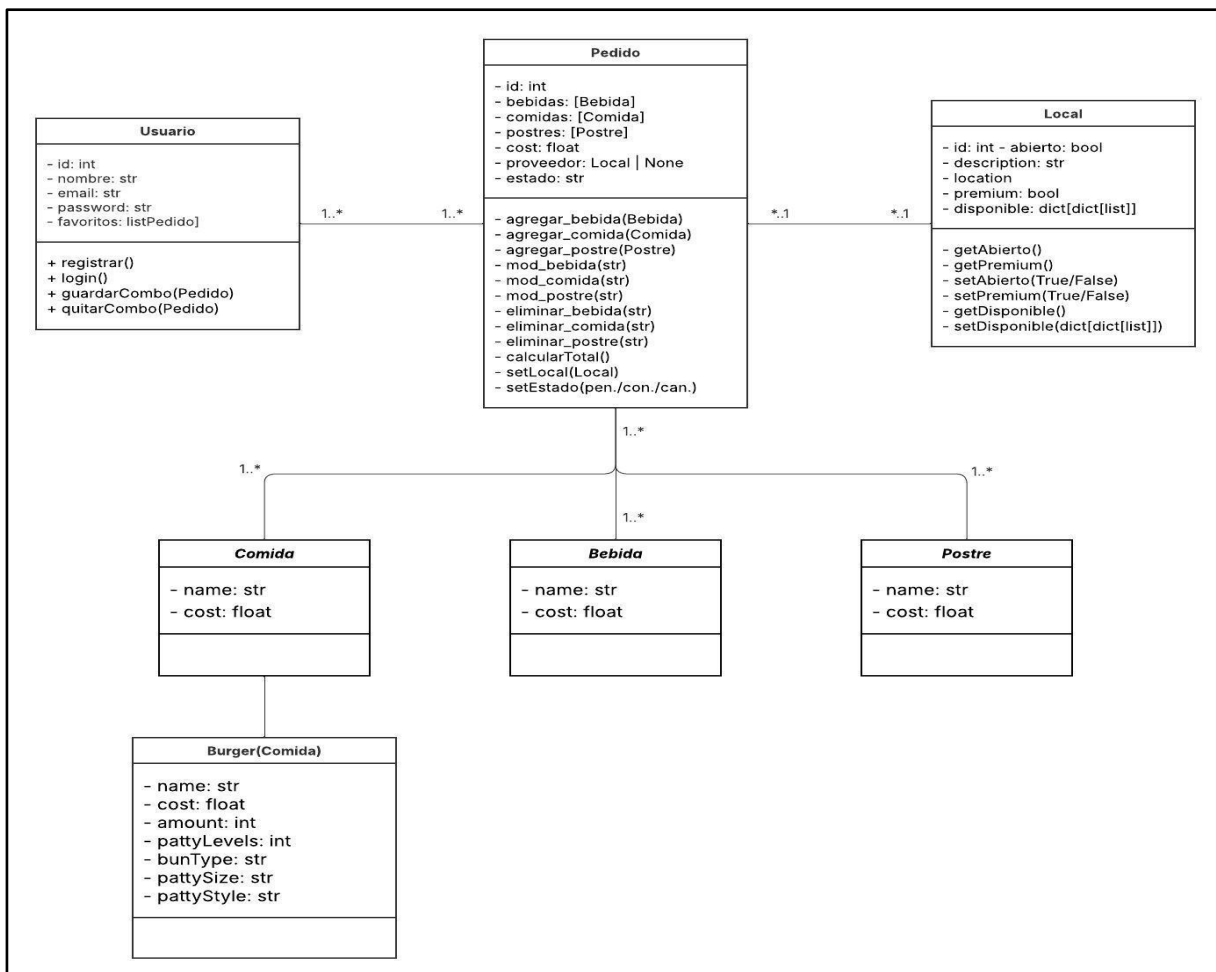
      1      *
Local ----- Bebida
      1      *

```

Local ----- Comida

Local 1 ----- * Postre

```
+-----+
| Local |
+-----+
| - id: int
| - abierto: bool
| - description: str
| - location
| - premium: bool
| - disponible: dict[dict[list]] // {"bebida": {...}, {...}}
+-----+
| - getAbierto()
| - getPremium()
| - setAbierto(True/False)
| - setPremium(True/False)
| - getDisponible()
| - setDisponible(dict[dict[list]])
+-----+
```



3.7. Patrones de diseño aplicados

Patrón	Aplicación en el proyecto
Decorator	Para crear objetos <code>Producto</code> (Base, Ingrediente, Bebida, etc.) dinámicamente según tipo.
Observer	Para notificar a locales sobre nuevos pedidos o actualizaciones en stock.
Singleton	Para manejar la conexión a base de datos o servicios compartidos.
Strategy (futuro)	Para calcular el precio total según distintas estrategias (descuentos, combos especiales, promociones).
MVC / MVVM	Estructura general: Controladores → Servicios → Repositorios → Modelos.

El MVP incluirá:

- Autenticación básica de usuarios.
- Creación de combos con validación simple.
- CRUD de locales y productos.
- Recomendaciones por cercanía y coincidencia de ingredientes.
- Persistencia en base relacional.
- UI web simple para probar el flujo completo

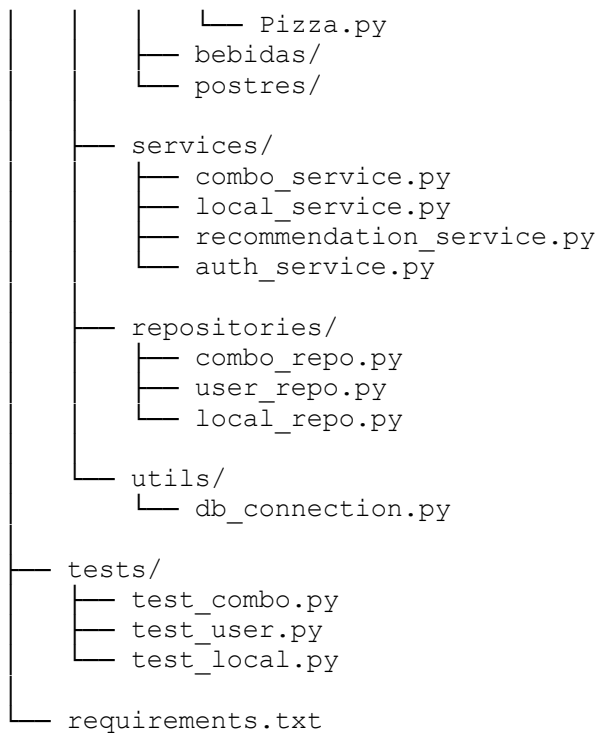
4) Implementación

Esta es la de implementación, donde pasamos del diseño a un prototipo funcional básico con clases, funciones *stub* (vacías) y algunos tests iniciales de TDD.

A continuación, desarrollamos una estructura típica de proyecto Python (API REST), siguiendo el diseño del MVP que definimos antes.

4.1. Estructura de directorios (prototipo)

```
combobuilder/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── controllers/
│   │   ├── combo_controller.py
│   │   ├── local_controller.py
│   │   ├── user_controller.py
│   │   └── order_controller.py
│   └── models/
│       ├── pedido.py
│       ├── user.py
│       ├── local.py
│       ├── product.py
│       └── comidas/
│           └── Burger.py
```



4.2. Esqueleto del código (clases y métodos vacíos)

app/models/usuario.py

```
class Usuario:
    def __init__(self, id: int, nombre: str, email: str, password:
str):
        self.id = id
        self.nombre = nombre
        self.email = email
        self.password = password
        self.favorites = []

    def registrar(self):
        pass

    def login(self):
        pass

    def guardarCombo(self, pedido):
        pass

    def quitarCombo(self, pedido):
        pass
```

app/models/usuario.py

```
class Pedido:
    def __init__(self, id: int):
        self.id = id
        self.bebidas = []
        self.comidas = []
```

```

        self.postres = []
        self.cost = 0.0
        self.proveedor = None
        self.estado = "pendiente" # pendiente / confirmado /
cancelado

    def agregar_bebida(self, bebida):
        pass

    def agregar_comida(self, comida):
        pass

    def agregar_postre(self, postre):
        pass

    def mod_bebida(self, nombre: str):
        pass

    def mod_comida(self, nombre: str):
        pass

    def mod_postre(self, nombre: str):
        pass

    def eliminar_bebida(self, nombre: str):
        pass

    def eliminar_comida(self, nombre: str):
        pass

    def eliminar_postre(self, nombre: str):
        pass

    def calcularTotal(self):
        pass

    def setLocal(self, local):
        pass

    def setEstado(self, estado: str):
        pass

```

app/models/usuario.py

```

class Local:
    def __init__(self, id: int, abierto: bool, description: str,
location, premium: bool, disponible: dict):
        self.id = id
        self.abierto = abierto
        self.description = description
        self.location = location
        self.premium = premium
        self.disponible = disponible # {"bebida": {...}, ...}

    def getAbierto(self):
        pass

    def getPremium(self):
        pass

```

```
- Fase 0:                                - Inicialización del Pedido
pedido1 = Pedido()

- Fase 1:                                - Inicialización de la Comida
comidal = Burger()

- Fase 2:                                - Personalización de la comida
comidal.set_pattySize(large)

comidal.set_pattyStyle(smash)
```

```
comida1.set_bunType(brioche)
```

```
CheddarCheese(comida1)
```

```
Bacon(comida1)
```

```
comida1.set_pattyLevels(3)
```

```
OnionsCaramelized(comida1)
```

```
comida1.set_amount(2)
```

```
pedido1.agregar_comida(comida1)
```

- Fase 3:

- Recomendación de Locales

```
validarLocales(pedido1)           {= [McDonalds, Wendys, BigPons]}
```

```
pedido1.setLocal(McDonalds)
```

```
pedido1.calcularTotal()           {= 11800.00}
```

```
McDonalds.get_description()
```

```
pedido1.setLocal(BigPons)
```

```
pedido1.calcularTotal()           {= 17500.00}
```

```
BigPons.get_description()
```

```
pedido1.setLocal(Wendys)
```

```
pedido1.calcularTotal()           {= 14000.00}
```

```
Wendys.get_description()
```

```
"continuar pidiendo en el local"  ->   Fase 1 (limitado a solo lo  
que ofrece Wendys)
```

- Fase 1:

```
bebida1 = Agua()
```

- Fase 2:

```
bebida1.set_size(large)
```

```
pedido1.agregar_bebida(bebida1)
```

- Fase 3:

```
pedido1.eliminar_bebida(bebida1)
```

- Fase 1:

```
bebida2 = Coca()
```

- Fase 2:


```
bebida2.set_size(medium)
```

```
bebida2.set_amount(2)
```

```
pedido1.agregar_bebida(bebida2)
```

- Fase 3:

```
"terminar pedido" -> Fase 4
```

- Fase 4:

```
(sucede el pago)
```

```
pedido1.setEstado(confirmado)
```

```
guardarCombo(pedido1)
```

```
// el usuario recibe su comida
```